Neural Architecture Search

Chapter Summary

Jovita Lukasik

What is NAS and Why Does It Matter?

Manual architecture design is slow, expert-dependent, and hard to reproduce.
NAS automates the search for high-performing architectures $a \in \mathcal{A}$.

Three interacting components:

Search space $\mathcal{A}$	Which architectures can be found? Encodes human prior knowledge.
Search strategy	Which architecture to evaluate next?
Performance estimation	How to evaluate $c(a)$ cheaply?

Applications: standard accuracy, hardware-aware NAS (latency per device), robustness-aware NAS (clean vs. adversarial accuracy). All can be cast as single- or multi-objective optimisation.

Search Spaces & Search Strategies

Search space types (from least to most expressive):

- **Chain-structured**: sequential layers, operation type per layer – simple baseline
- **Cell-based** (NASNet, DARTS): search for a small repeated cell; fixed macro structure – *most popular*: compact, transferable across datasets
- **Macro/Hierarchical**: full DAG topology at one or multiple levels – expressive but hard

Search strategies:

- **Random / local search**: simple; surprisingly competitive on well-engineered spaces
- **Evolutionary algorithms** (e.g., aging evolution): mutate/crossover a population; aging drops oldest not worst – adaptive, handles discrete spaces
- **Bayesian optimisation**: encode architecture as categorical HPs; tree-based surrogates (TPE, SMAC) enable joint architecture + HP search

Take-away: search space design has a *larger* impact than the choice of search strategy.

Performance Estimation & Speedup Techniques

Ground truth (full training from scratch) costs up to 800 GPU hours for 20k architectures.

Three speedup families:

Technique	Idea	Caveat
Performance predictors	Dataset of evaluated archs + encoding + model (GNN, RF) $\rightarrow$ predict $c(a)$	Needs labelled training data
Learning curve extrapolation	Train $t_{\text{early}} \ll T$ epochs; fit & extrapolate; terminate poor candidates early	Fails for unusual dynamics
Zero-cost proxies (ZCP)	Score *untrained* network with one forward/backward pass	Weak, inconsistent correlation; #params often competitive

Predictor-guided search: embed a surrogate inside BO or evolution, evaluate only top-k predicted candidates.

One-Shot NAS & DARTS

One-shot / weight-sharing NAS: build a *supernet* – a single over-parameterised DAG containing every architecture as a subgraph. Train it once; candidate architectures inherit weights for free-cost evaluation.

- **DropPath**: randomly zero edges per mini-batch to prevent weight co-adaptation
- **Sampling strategies**: uniform (RSWS), policy-based (ENAS), or decoupled training + evolutionary search (SPOS)
- Key challenge: ranking correlation between supernet and separately trained models is often low

DARTS (Liu et al. 2019): make the search *differentiable* by replacing hard operation choices with a continuous mixture weighted by learned α. Discretise at the end.
Issues: sensitive to HPs; collapses to degenerate architectures; high memory cost.

Chapter Summary

1. NAS automates architecture design through three components – **search space, strategy, and performance estimation** – all of which must be designed carefully
2. **Cell-based spaces** dominate in practice; search space design matters more than strategy choice – random search is a strong baseline
3. **Performance predictors, LC extrapolation, and ZCPs** reduce evaluation cost from days to minutes; ZCPs are fast but unreliable
4. **One-shot NAS** avoids retraining via weight sharing; ranking correlation with stand-alone models is the main unsolved challenge
5. **DARTS** enables gradient-based search but is brittle; **expressive CFG-based spaces** (einspace) point toward genuinely novel architectures beyond ConvNets

⇝ The search space is the biggest lever in NAS. The best search strategy cannot compensate for a poorly designed space.